

Instituto Tecnológico de San Luis Potosí

Ingeniería en Sistemas Computacionales

Lenguajes de Interfaz

11:00-12:00

**Herramientas para la programación de
dispositivos.**

Alumna:

Venegas Ordaz Fátima Vianney

Lugar y Fecha:

Soledad de Graciano Sánchez, San Luis Potosí, 11 de abril de 2026

Índice

Introducción.	3
Herramientas para la programación de dispositivos y su código.	4
STM32CubeIDE.	4
SparkFun RedBoard Artemis.....	5
MPLAB [®] X Entorno de desarrollo integrado (IDE).....	7
Conclusión.	9
Referencias.....	10

Introducción.

EL objetivo del presente documentos es ver el desarrollo de sistemas y como la programación de dispositivos han ido evolucionado más allá de las plataformas de aprendizaje básicas. Si bien Arduino es el estándar para prototipado rápido, el entorno industrial y profesional exige herramientas con mayor control sobre los recursos del hardware.

Herramientas para la programación de dispositivos y su código.

STM32CubeIDE.

STM32CubeIDE es un entorno para editar, compilar y depurar código. Los usuarios suelen utilizarlo junto con otras herramientas del ecosistema STM32Cube, como la herramienta de configuración gráfica STM32CubeMX. También es compatible con diversas herramientas de soluciones verticales, como TouchGFXDesigner, STM32Cube AI Studio y Motor Control Workbench.

Utiliza una arquitectura de capas. Tú programas sobre una capa llamada HAL (Hardware Abstraction Layer), que traduce tus comandos a lenguaje de registros que el chip entiende. (STMicroelectronics, 2024)

Código demostración en C.

```
#include "main.h"

// Función principal del sistema

int main(void) {

    // 1. Inicialización de la capa de abstracción de hardware

    HAL_Init();

    // 2. Configuración de los pines (GPIO)

    // En STM32, los periféricos están apagados por defecto para ahorrar
    energía

    __HAL_RCC_GPIOA_CLK_ENABLE(); // Habilitar reloj del puerto A

    GPIO_InitTypeDef GPIO_InitStruct = {0};

    GPIO_InitStruct.Pin = GPIO_PIN_5;
```

```

GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP; // Salida Push-Pull

GPIO_InitStruct.Pull = GPIO_NOPULL;

GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;

HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);


// 3. Bucle infinito de control del dispositivo

while (1) {

    HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5); // Alternar estado físico del
pin

    HAL_Delay(500); // Retraso basado en el SysTick del procesador

}

} (STMicroelectronics, 2024)

```

SparkFun RedBoard Artemis.

El SparkFun Artemis es un módulo increíble. ¡Una gran cantidad de funcionalidades en un espacio reducido de 10x15 mm! Pero lo que realmente lo hace potente es la capacidad de escribir bocetos rápidamente y crear proyectos usando solo código Arduino. (SparkFun Electronics, s.f.)

A diferencia de los procesadores básicos, el Artemis utiliza un núcleo ARM Cortex-M4F con una velocidad de hasta 96MHz. Lo que lo hace especial para la programación de dispositivos es su capacidad de "morder" ráfagas de datos de sensores y procesarlas con IA sin necesidad de conectarse a la nube.

Se programa principalmente en C y C++. También soporta TensorFlow Lite, permitiendo cargar modelos de inteligencia artificial (como reconocimiento de voz o gestos) directamente en el chip.

Este código muestra cómo se inicializa el hardware para trabajar con TensorFlow Lite Micro, algo imposible en un Arduino convencional. (Torrente, 2023)

Código en c++.

```
#include "tensorflow/lite/micro/all_ops_resolver.h"

#include "tensorflow/lite/micro/micro_interpreter.h"

#include "tensorflow/lite/schema/schema_generated.h"


// Definición del área de memoria para el modelo de IA (Tensor Arena)

const int kTensorArenaSize = 10 * 1024;

uint8_t tensor_arena[kTensorArenaSize];


void setup_artemis_ia() {

    // 1. Cargar el modelo desde la memoria Flash del dispositivo

    const tflite::Model* model = tflite::GetModel(g_model_data);


    // 2. Configurar el intérprete para ejecutar el modelo en el núcleo Cortex-

static tflite::MicroInterpreter interpreter(

    model, resolver, tensor_arena, kTensorArenaSize, error_reporter);


    // 3. Asignar memoria física del hardware a los tensores

    interpreter.AllocateTensors();

}
```

M4

```

void loop_ia() {

    // Ejecutar inferencia de IA localmente en el dispositivo

    interpreter.Invoke();

}

```

MPLAB® X Entorno de desarrollo integrado (IDE)

MPLAB X Integrated Development Environment (IDE) es un programa de software ampliable y altamente configurable que incorpora potentes herramientas para ayudarle a descubrir, configurar, desarrollar, depurar y validar diseños embebidos para la mayoría de nuestros microcontroladores y controladores de señales digitales. MPLAB X IDE funciona a la perfección con el ecosistema de software y herramientas de desarrollo de MPLAB, muchas de las cuales son totalmente gratuitas.

Se utiliza un flujo de **Compilación y Depuración (Debug)**. El programador debe conocer el mapa de memoria del dispositivo.

En esta herramienta, el programador debe configurar los "Configuration Bits" (fusibles del hardware) antes de iniciar, lo cual define cómo se comportará el oscilador físico y los sistemas de protección del chip. (Microchip Technology, 2024)

Código de demostración en C.

```

// Configuración de bits (Fusibles del hardware)

#pragma config FOSC = INTOSCIO // Oscilador interno

#pragma config WDTE = OFF      // Perro guardián desactivado (Watchdog
Timer)

#pragma config PWRTE = ON      // Temporizador de encendido activado


#include <xc.h>                // Biblioteca principal del compilador XC8

```

```
#define _XTAL_FREQ 4000000 // Definir frecuencia de 4MHz para retardos
```

```
void main(void) {
```

```
    // 1. Configuración de registros de dirección (TRIS)
```

```
    // TRISB = 0 significa que todo el Puerto B es SALIDA
```

```
    TRISB = 0x00;
```

```
    // 2. Inicializar el estado de los pines (LAT)
```

```
    LATB = 0x00;
```

```
    while(1) {
```

```
        // 3. Manipulación directa de los niveles lógicos del hardware
```

```
        LATBbits.LATB0 = 1; // Poner el pin RB0 en estado alto (5V)
```

```
        __delay_ms(500);    // Retraso de medio segundo
```

```
        LATBbits.LATB0 = 0; // Poner el pin RB0 en estado bajo (0V)
```

```
        __delay_ms(500);
```

```
    }
```

```
}
```

Conclusión.

Al finalizar esta investigación, es posible concluir que la elección de una herramienta de programación depende directamente de la complejidad del dispositivo y los requisitos de eficiencia del proyecto. El dominio de estas herramientas profesionales es fundamental para cualquier ingeniero que busque diseñar soluciones tecnológicas.

Referencias.

Microchip Technology. (2024). MPLAB® X Integrated Development Environment (IDE). Recuperado de <https://www.microchip.com/en-us/tools-resources/develop/mplab-x-ide>

SparkFun Electronics. (s.f.). Artemis Development with Arduino: All tutorials. Recuperado de <https://learn.sparkfun.com/tutorials/artemis-development-with-arduino/all>

STMicroelectronics. (2024). STM32CubeIDE: Integrated Development Environment for STM32. Recuperado de <https://www.st.com/en/development-tools/stm32cubeide.html>

Torrente, A. (2023). Alternativas a Arduino: Microcontroladores de alto rendimiento y bajo consumo. Recuperado de <https://descubrearduino.com/alternativas-arduino/>